

本节讲解的例题及其囊括的知识点，如表4-6所示。

表4-6 树的经典问题与方法例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 4-27	HDU 2586	How far away	倍增 LCA	陈锋
例 4-28	POJ 1741	Tree	树的点分治；路径统计	陈锋
例 4-29	UVa 12161	Ironman Race in Treeland	树的点分治；路径统计	陈锋
例 4-30	UVa 1674	Lightning Energy Report	树链剖分；维护边权；树上差分	陈锋
例 4-31	NC 17882	软件包管理器	树链剖分；维护点权；子树点权	陈锋
例 4-32	IPSC 2009 Problem L	Let there be rainbows	基于迭代的树链剖分	陈锋

4.5 动态树与LCT

LCT是解决动态树问题的强力工具，本节提供一些常见的LCT应用题目以及代码实现。

例4-33 【LCT；维护连通性】洞穴勘测（Cave, SDOI 2008，牛客NC 20311）

给出 n ($n \leq 10^4$) 个独立的点以及 m ($m \leq 2 \times 10^5$) 个操作，操作分为以下3种类型。

- ☐ Connect $u\ v$: 在 u, v 两点之间连接一条边。
- ☐ Destroy $u\ v$: 删除在 u, v 两点之间的边（应保证存在这样的一条边）。
- ☐ Query $u\ v$: 询问 u, v 两点是否连通。

保证在任何时刻，图的形态都是一个森林，输出每个Query操作的结果（Yes或者No）。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
const int NN = 1e5 + 4;
template <int SZ>
struct LCT {
    int ch[SZ][2], fa[SZ], rev[SZ];
    void clear(int x) { ch[x][0] = ch[x][1] = fa[x] = rev[x] = 0; }
    // x 是否为辅助树上父亲的右儿子
    inline int is_right_ch(int x) { return ch[fa[x]][1] == x; }
    // x 是否为辅助树根
    inline int is_root(int x) { return ch[fa[x]][0] != x && ch[fa[x]][1] != x; }
    void pushdown(int x) {
        if (rev[x] == 0) return;
        int lx = ch[x][0], rx = ch[x][1];
        if (lx) swap(ch[lx][0], ch[lx][1]), rev[lx] ^= 1;
        if (rx) swap(ch[rx][0], ch[rx][1]), rev[rx] ^= 1;
        rev[x] = 0;
    }
    void pushup(int x) {
        if (!is_root(x)) pushup(fa[x]);
        pushdown(x);
    }
};

```

```

}
void rotate_up(int x) { // 将x向上旋转一级
    int y = fa[x], z = fa[y], chx = is_right_ch(x), chy = is_right_ch(y),
        &t = ch[x][chx ^ 1]; // t在x,y之间,但是t-x, x-y方向相反
    fa[x] = z;
    if (!is_root(y)) ch[z][chy] = x; // x,y在z的同一侧
    ch[y][chx] = t, fa[t] = y, t = y, fa[y] = x; // 保证t依然在x,y之间
}
void splay(int x) {
    pushup(x); // x到树根路径上所有点的深度相对关系都要反转
    for (int f = fa[x]; f = fa[x], !is_root(x); rotate_up(x))
        if (!is_root(f)) rotate_up(is_right_ch(x) == is_right_ch(f) ? f : x);
}
void access(int x) { // 将root-x变成首选边
    for (int f = 0; x; f = x, x = fa[x]) splay(x), ch[x][1] = f;
}
void make_root(int x) { // 将x变为树根
    access(x), splay(x), swap(ch[x][0], ch[x][1]), rev[x] ^= 1;
}
void split(int x, int y) { make_root(x), access(y), splay(y); }
int find_root(int x) { // x所在树的树根
    access(x), splay(x);
    while (ch[x][0]) x = ch[x][0];
    splay(x);
    return x;
}
void cut(int x, int y) {
    split(x, y); // x是y在辅助树中的左孩子且要求x,y相邻
    if (ch[y][0] == x && !ch[x][1]) ch[y][0] = fa[x] = 0;
}

void link(int x, int y) {
    if (find_root(x) != find_root(y)) make_root(x), fa[x] = y;
}
};
LCT<NN> st;
int main() {
    int n, q, x, y;
    char op[16];
    scanf("%d%d", &n, &q);
    while (q--) {
        scanf("%s%d%d", op, &x, &y);
        switch (op[0]) {
            case 'Q':
                puts(st.find_root(x) == st.find_root(y) ? "Yes" : "No");
                break;
            case 'C':
                st.link(x, y);
                break;
        }
    }
}

```

```

        case 'D':
            st.cut(x, y);
            break;
        default:
            break;
    }
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.8 节例题 41
*/

```

例4-34 【LCT；维护路径信息】快乐涂色（Happy Painting, UVa 11994）

n 个结点组成了若干棵有根树，树中的每条边都有一个特定的颜色。你的任务是执行 m 条操作，输出结果。操作一共有如下3种。

- ❑ 1 $x\ y\ c$: 把 x 的父结点改成 y 。如果 $x=y$ 或者 x 是 y 的祖先，则忽略这条指令；否则删除 x 和它原先父结点之间的边，而新边的颜色为 c 。
- ❑ 2 $x\ y\ c$: 把 x 和 y 简单路径上的所有边涂成颜色 c 。如果 x 和 y 之间没有路径，则忽略此指令。
- ❑ 3 $x\ y$: 统计 x 和 y 简单路径上的边数，以及这些边一共有多少种颜色。

每组数据第一行为 n 和 m （ $1 \leq n \leq 50\ 000$ ， $1 \leq m \leq 200\ 000$ ），然后是每个结点的父结点编号和该结点与其父结点之间的边的颜色（对于根结点，父结点编号为0，且“与父结点之间的边的颜色”无意义）。接下来是 m 条指令。对于所有指令， $1 \leq x, y \leq n$ ；对于类型2指令， $1 \leq c \leq 30$ 。结点编号为 $1 \sim n$ ，颜色编号为 $1 \sim 30$ 。对于每个类型3指令，输出对应的结果。

【代码实现】

```

// 魏子豪 陈锋
#include <bits/stdc++.h>
const int NN = 1000005;
using namespace std;

template <int SZ>
struct LCT {
    // clr[x]: x-fa[x]之间的边权
    int ch[SZ][2], fa[SZ], rev[SZ], size[SZ], clr[SZ], set[SZ], mark[SZ];
    void init(int x) {
        ch[x][0] = ch[x][1] = fa[x] = 0;
        rev[x] = size[x] = clr[x] = set[x] = mark[x] = 0;
    }
    int is_right_ch(int x) { return ch[fa[x]][1] == x; }
    bool is_root(int x) { return ch[fa[x]][0] != x && ch[fa[x]][1] != x; }
    void maintain(int x) {
        int &sx = set[x], &sz = size[x], ls = ch[x][0], rs = ch[x][1];
        sx = 0, sz = 1;
        if (ls) sx |= set[ls] | (1 << clr[ls]), sz += size[ls];
    }
};

```

```

    if (rs) sx |= set[rs] | (1 << clr[rs]), sz += size[rs];
}
void rotate_up(int x) { // 旋转和无根树的直接旋转不同
    int y = fa[x], d = is_right_ch(x), &t = ch[y][d], z = fa[y],
        cy_bak = clr[y], &cx = clr[x], &cy = clr[y];
    fa[x] = z; // 辅助树中深度关系: t 在 x,y 之间, x,y 在 z 同一侧
    if (!is_root(y)) ch[z][is_right_ch(y)] = x;
    t = ch[x][d ^ 1];
    if (t) // 边权挂在更深的点上
        fa[t] = y, cy = clr[t], clr[t] = cx, cx = cy_bak;
    else
        swap(cx, clr[y]);
    ch[x][d ^ 1] = y, fa[y] = x;
    maintain(y), maintain(x);
}
void pushup(int x) {
    if (!is_root(x)) pushup(fa[x]);
    pushdown(x);
}
void pushdown(int x) { // 将翻转标记和染色标记下传
    int ls = ch[x][0], rs = ch[x][1], &mk = mark[x];
    if (mk) {
        if (ls) clr[ls] = mark[ls] = mk, set[ls] = set[x];
        if (rs) clr[rs] = mark[rs] = mk, set[rs] = set[x];
        mk = 0;
    }
    if (rev[x]) {
        swap(ch[x][0], ch[x][1]);
        if (ls) rev[ls] ^= 1;
        if (rs) rev[rs] ^= 1;
        rev[x] = 0;
    }
}
void splay(int x) {
    pushup(x);
    for (int f = fa[x]; f = fa[x], !is_root(x); rotate_up(x))
        if (!is_root(f)) rotate_up(is_right_ch(x) == is_right_ch(f) ? f : x);
    maintain(x);
}
void access(int x) {
    for (int last = 0; x; x = fa[x])
        splay(x), ch[x][1] = last, last = x, maintain(last);
}
int find_root(int x) {
    access(x), splay(x);
    while (ch[x][0]) x = ch[x][0];
    splay(x);
    return x;
}

```

```

void make_root(int x) { access(x), splay(x), rev[x] ^= 1; }
void split(int x, int y) { make_root(x), access(y), splay(y); }
void cut(int x) {
    access(x), splay(x); // x 到 Root 拉成一条链, 将 x 旋转至辅助树根
    int& ls = ch[x][0];
    if (ls) fa[ls] = 0, clr[ls] = 0, ls = 0; // 左儿子就是树中 x 之父, 直接断掉
}
void link(int x, int y, int color) { // 实际数据中没有 x 为 y 父亲的情况
    access(y), splay(x);
    cut(x), fa[x] = y, clr[x] = color; // 直接连边
}
// 使 x 成为所在树的根, 然后将 x-y 的路径上的所有点加入一个 Splay 中
void paint(int x, int y, int c) {
    int rx = find_root(x);
    if (rx != find_root(y)) return; // x,y 不连通
    // 根 x-y 是当前的首选路径, v 是 splay 根, splay 中只有 x-y
    split(x, y), set[y] = 1 << c, mark[y] = c;
    make_root(rx); // 还原树根
}
void query(int x, int y, int& sz, int& cc) {
    int rx = find_root(x);
    sz = 0, cc = 0;
    if (rx != find_root(y)) return; // 如果 u 和 v 不在同一棵树, 直接输出 0 0
    split(x, y);
    for (int k = set[y]; k; k >>= 1) cc += k & 1; // 统计有几种不同的颜色
    sz = size[y] - 1;
    make_root(rx); // 还原树根
}
};

```

LCT<NN> T;

```

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    for (int n, m; cin >> n >> m; ) {
        for (int i = 1; i <= n; i++) T.init(i);
        for (int i = 1, v; i <= n; i++) cin >> v, T.fa[i] = v;
        for (int i = 1, v; i <= n; i++) {
            cin >> v;
            if (T.fa[i]) T.clr[i] = v; // 将边权放在深度较深的点上
        }
        for (int i = 1, op, u, v, c; i <= m; i++) {
            cin >> op >> u >> v;
            switch (op) {
                case 1:
                    cin >> c; // x--c-->y
                    if (u != v) T.link(u, v, c);
                    break;
                case 2:
                    cin >> c, T.paint(u, v, c);
            }
        }
    }
}

```

```

        break;
    case 3:
        int sz, cc;
        T.query(u, v, sz, cc);
        printf("%d %d\n", sz, cc);
        break;
    }
}
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.8 节例题 42
注意：本题中 LCT 如何借鉴线段树中的 pushdown 以及 maintian 函数，实现标记操作
同类题目：航线规划，AHOI 2005，牛客 NC 19871
Elephant, IOI 2011, Codechef ELPHANT
*/

```

例4-35 【LCT；维护MST】大厨和图上查询（Chef and Graph Queries, Codechef-GERALD 07）

大厨有一个无向图 G ，顶点标号为 $1 \sim N$ ，边标号为 $1 \sim M$ 。给出 Q 对询问 L_i, R_i ($1 \leq L_i \leq R_i \leq M$)，对于每对询问，大厨想知道当仅保留编号 X 满足 $L_i \leq X \leq R_i$ 所在的边时，图 G 中有多少个连通块。请帮助大厨回答这些询问。其中， $1 \leq N, M, Q \leq 2 \times 10^5$ 。

注意：数据可能包含自环和重边。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;

template <int SZ>
struct LCT {
    int ch[SZ][2], fa[SZ], minw[SZ]; // 最小点权
    bool rev[SZ];
    inline int& ls(int x) { return ch[x][0]; }
    inline int& rs(int x) { return ch[x][1]; }
    inline void reverse(int x) { rev[x] ^= 1, swap(ls(x), rs(x)); }
    inline void maintain(int x) {
        minw[x] = min(x, min(minw[ls(x)], minw[rs(x)]));
    }
    inline void pushdown(int x) {
        if (rev[x]) reverse(ls(x)), reverse(rs(x)), rev[x] = false;
    }
    inline bool isroot(int x) { return ls(fa[x]) != x && rs(fa[x]) != x; }
    inline int isright(int x) {
        return rs(fa[x]) == x;
    }
    // x 是否为 Splay 上父亲的右儿子
    void rotate(int x) {

```

```

    int y = fa[x], z = fa[y], k = isright(x), &t = ch[x][k ^ 1];
    if (!isroot(y)) ch[z][isright(y)] = x; // x,y 在 z 的同一侧
    ch[y][k] = t, fa[t] = y; // 设置 y,t 之间的关系, x,t 都在 y 的同一侧
    t = y, fa[y] = x, fa[x] = z; // x-y, y-t 方向相反
    maintain(y), maintain(x);
}

void pushup(int x) {
    if (!isroot(x)) pushup(fa[x]);
    pushdown(x);
}

void splay(int x) {
    pushup(x);
    while (!isroot(x)) {
        int y = fa[x];
        if (!isroot(y)) rotate(isright(y) == isright(x) ? x : y);
        rotate(x);
    }
}

void access(int x) {
    for (int t = 0; x; t = x, x = fa[x]) splay(x), rs(x) = t, maintain(x);
}

void makeroot(int x) { access(x), splay(x), reverse(x); }

void link(int x, int y) { makeroot(x), fa[x] = y; }

void cut(int x, int y) {
    makeroot(x), access(y), splay(y);
    ls(y) = fa[x] = 0;
    maintain(y);
}

void split(int x, int y) { makeroot(x), access(y), splay(y); }

int findroot(int x) {
    access(x), splay(x);
    while (ls(x)) pushdown(x), x = ls(x);
    splay(x);
    return x;
}

void init(int sz) {
    minw[0] = 1e9;
    assert(sz < SZ);
    for (int i = 1; i <= sz; i++) // LCT 初始化
        minw[i] = i, ch[i][0] = ch[i][1] = fa[i] = 0, rev[i] = 0;
}

};

template <int SZ>
struct BIT {
    int C[SZ], n;
    void init(int sz) { assert(sz + 1 < SZ), fill_n(C, sz + 1, 0), this->n = sz; }
    inline int lowbit(int x) { return x & -x; }
    void add(int x, int v) {

```

```

    while (x <= n) C[x] += v, x += lowbit(x);
}
int sum(int x) {
    int ret = 0;
    while (x) ret += C[x], x -= lowbit(x);
    return ret;
}
};
const int NN = 2e5 + 4;
BIT<NN> S;
LCT<NN * 2> lct;
int QL[NN], Ans[NN], EU[NN], EV[NN];
vector<int> EQ[NN];

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int T;
    cin >> T;
    for (int t = 0, n, m, q; t < T; t++) {
        cin >> n >> m >> q;
        for (int i = 1; i <= m; i++) {
            int &u = EU[i], &v = EV[i];
            cin >> u >> v, u += m, v += m, EQ[i].clear();
        }
        S.init(m), lct.init(m + n);
        for (int i = 1, qr; i <= q; i++) cin >> QL[i] >> qr, EQ[qr].push_back(i);
        for (int i = 1; i <= m; i++) {
            int u = EU[i], v = EV[i];
            if (lct.findroot(u) == lct.findroot(v)) { // u,v 已经连通
                lct.split(u, v);
                int e = lct.minw[v]; // v 所在分量的最小边权
                if (e < i) { // 边 i 比 x 大, 删除 x
                    lct.cut(e, EU[e]), lct.cut(e, EV[e]), S.add(e, -1); // 删除边 x
                    lct.link(i, u), lct.link(i, v), S.add(i, 1); // 加入边 i
                }
            } else
                lct.link(u, i), lct.link(v, i), S.add(i, 1); // 加入边 i

            for (size_t xi = 0; xi < EQ[i].size(); xi++)
                Ans[EQ[i][xi]] = n - (S.sum(i) - S.sum(QL[EQ[i][xi]] - 1));
        }
        for (int i = 1; i <= q; i++) cout << Ans[i] << endl;
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 3.8 节例题 43
注意: 本题中如何使用一些子函数简化 LCT 各种操作实现
同类题目: 魔法森林, NOI 2014, 牛客 NC 17858
*/

```